Estruturas de repetições – utilizadas para executar a mesma parte de um programa várias vezes.

teste.py - C:/Users/willi/OneDrive/Área de Trabalho/teste.py (3.13.12)

File Edit Format Run Options Window Help

```
#programa imprimi 3 números na tela
print (1)
print (2)
print (3)
```

teste2.py - C:/Users/willi/OneDrive/Área de Trabalho/teste2.py (3.13.12)

File Edit Format Run Options Window Help

```
x = 1
print (x)
x = 2
print (x)
x = 3
print (x)
```

teste2.py - C:/Users/willi/OneDrive/Área de Trabalho/teste2.py (3.13.12)

File Edit Format Run Options Window Help

```
x = 1
print (x)
x = x + 1
print (x)
x = x + 1
print (x)
```

Mas e se você precisasse escrever 100 números? Poderia usar a estrutura de repetição – **while** – que repete um bloco enquanto a condição for verdadeira.

Formato:

while *<condição>* *# condição = expressão lógica*
    bloco

Considere o programa anterior.

teste2.py - C:/Users/willi/OneDrive/Área de Trabalho/teste2.py (3.13.12)

File Edit Format Run Options Window Help

```
# programa imprimi 3 números usando while
x = 1
while x <=3:
    print (x)
    x = x + 1

# IMPORTANTE: PODEMOS ENTENDER A CONDIÇÃO while COMO O if,
# COM A DIFERENÇA DE QUE SE A CONDIÇÃO FOR VERDADEIRA, AS
#LINHAS 3 E 4 SERÃO REPETIDAS (BLOCO) ENQUANT A AVALIAÇÃO
# DA CONDIÇÃO FOR VERDADEIRA.
```

Contadores – são estruturas de repetição em que utilizamos condições com mais de uma variável.

teste2.py - C:/Users/willi/OneDrive/Área de Trabalho/teste2.py (3.13.12)

File Edit Format Run Options Window Help

```python
# programa que imprimi os números inteiros entre 1 e um valor digitado pelo usuário.
# no programa o último número a imprimir é informado pelo usuário.

fim = int(input("Digite o último número a imprimir:"))

x = 1
while x <= fim:
    print (x)
    x = x + 1

# uso prático de estruturas de repetições
#1 Em uma fábrica, pode-se registrar peças produzidas até certa quantidade.
#2 Processar parcelas ou prestações, mostrando as parcelas de um financiamento
#3 Repetir leituras de sensores ou medições em automação, como 20 leituras de
# temperatura, pressão, umidade etc.
```

teste2.py - C:/Users/willi/OneDrive/Área de Trabalho/teste2.py (3.13.12)

File Edit Format Run Options Window Help

```python
# programa que imprimi apenas os números pares usando if

fim = int(input("Digite o último número a imprimir:"))

x = 0
while x <= fim:
    if x % 2 == 0: # (resto da divisão de x por 2 é igual a zero)
        print (x)
    x = x + 1
```

teste2.py - C:/Users/willi/OneDrive/Área de Trabalho/teste2.py (3.13.12)

File Edit Format Run Options Window Help

```python
# programa que imprimi apenas os números pares - opção sem if

fim = int(input("Digite o último número a imprimir:"))

x = 0
while x <= fim:
    print (x)
    x = x + 2
```

Contadores podem ser utilizados com condições dentro dos programas.

teste2.py - C:/Users/willi/OneDrive/Área de Trabalho/teste2.py (3.13.12)

File Edit Format Run Options Window Help

```python
# programa que corrigi um teste de multiplaescolha com 3 questões. As respostas corretas
# são: 1. b; 2. a e 3. d. Um ponto para cada questão correta é atribuído.

pontos = 0
questão = 1

while questão <= 3:
    resposta = input(f"Resposta da questão {questão}: ")
    if questão == 1 and resposta == "b": # "b" or resposta == "B")
        pontos = pontos + 1
    if questão == 2 and resposta == "a":
        pontos = pontos + 1
    if questão == 3 and resposta == "d":
        pontos = pontos + 1
    questão = questão + 1

print (f"O aluno fez {pontos} ponto(s)")
```

Acumuladores → aqui o valor adicionado é variável diferentemente dos acumuladores que é constante.

teste2.py - C:/Users/willi/OneDrive/Área de Trabalho/teste2.py (3.13.12)

File Edit Format Run Options Window Help

```python
# programa que calcule a soma de 10 números.

n = 1
soma = 0

while n <= 10:
    x = int(input(f"Digite o {n} número: "))
    soma = soma + x # acumulador
    n = n + 1       # contador

print (f"Soma: {soma}")
```

Operadores de atribuições especiais

Operador	Exemplo	Equivalência
+=	x += 1	x = x + 1
-=	y -= 1	y = y – 1
*=	c *= 2	c = c * 2
/=	d /= 2	d = d / 2
**=	e **= 2	e = e ** 2
//=	f //= 4	f = f // 4

Uso da instrução **break** para interromper a execução de while independente do valor atual de sua condição.

teste2.py - C:/Users/willi/OneDrive/Área de Trabalho/teste2.py (3.13.12)

File Edit Format Run Options Window Help

```
# programa leitura de valores até digitarmos 0 (zero).

s = 0

while True: #a condição while foi substituída por True
    v = int(input("Digite um número a somar ou 0 para sair:"))
    if v == 0:
        break
    s += v

print (s)
```

Exemplo proposto. Construa um programa que leia um valor e imprima a quantidade de cédulas necessárias para pagar esse mesmo valor. Para simplificar vamos trabalhar apenas com valores inteiros e com cédulas de R$ 50, R$ 20, R$ 10, R$ 5 e R$ 1.

Observação: Posteriormente faremos modificações no programa (exercícios 8, 9 e 10).

teste2.py - C:/Users/willi/OneDrive/Área de Trabalho/teste2.py (3.13.12)

File Edit Format Run Options Window Help

```python
# programa: contagem de cédulas

valor = int(input("Digite o valor a pagar:"))

cédulas = 0
atual = 50
apagar = valor

while True:
    if atual <= apagar:
        apagar -= atual
        cédulas += 1
    else:
        print(f"{cédulas} cédula(s) de R${atual}")
        if apagar == 0:
            break
        if atual == 50:
            atual = 20
        elif atual == 20:
            atual = 10
        elif atual == 10:
            atual = 5
        elif atual == 5:
            atual = 1
        cédulas = 0
```

Como o programa funciona na prática

Suponha que o usuário digite: 87
O processo será:

- **tenta com 50:**
  - usa 1 nota de 50
  - sobra 37
- tenta com 50 novamente:
  - não cabe mais
  - imprime 1 cédula(s) de R$50

  passa para 20
- tenta com 20:
  - usa 1 nota de 20
  - sobra 17
- tenta com 20 novamente:
  - não cabe
  - imprime 1 cédula(s) de R$20

  passa para 10
- tenta com 10:
  - usa 1 nota de 10
  - sobra 7

•imprime 1 cédula(s) de R$10

passa para 5

•usa 1 nota de 5

•sobra 2

•imprime 1 cédula(s) de R$5

passa para 1

•usa 2 notas de 1

•sobra 0

•imprime 2 cédula(s) de R$1

•termina

Saída: 1 cédula(s) de R$50
1 cédula(s) de R$20
1 cédula(s) de R$10
1 cédula(s) de R$5
2 cédula(s) de R$1

Exercícios:

1. Faça um programa para escrever contagem regressiva do lançamento de um foguete. O programa deve imprimir 10, 9, ..., 1, 0 e Fogo! na tela.
2. Escreva um programa para escrever os 10 primeiros múltiplos de 3.
3. Escreva um programa que leia dois números. Imprima o resultado da multiplicação do primeiro pelo segundo. Utilize apenas os operadores de soma e subtração para calcular o resultado. Lembre-se de que podemos entender a multiplicação de dois números como somas sucessivas de um deles.

4. Escreva um programa que leia dois números. Imprima a divisão inteira do primeiro pelo segundo, assim como o resto da divisão. Utilize apenas os operadores de soma e subtração para calcular o resultado. Lembre-se de que podemos entender a quociente da divisão de dois números como a quantidade de vezes que podemos retirar o divisor do dividendo.

5. Escreva um programa que pergunte o depósito inicial e a taxa de juros de uma poupança. Exiba os valores mês a mês para os 24 primeiros meses. Escreva o total ganho com juros no período.

6. Escreva um programa que pergunte o valor inicial de uma dívida e o juro mensal. Pergunte também o valor mensal que será pago. Imprima o número de meses para que a dívida seja paga, o total pago e o total de juros pagos.

7. Escreva um programa para controlar uma pequena máquina registradora. Você deve solicitar ao usuário que digite o código do produto e a quantidade comprada. Use a tabela de códigos para obter o preço de cada produto. Seu programa deve exibir o total de compras depois que o usuário digitar 0. Qualquer outro código deve gerar a mensagem de erro “Código inválido”.

Código	Preço (R$)
1	0,50
2	1,00
3	4,00
4	7,00
5	8,00

8. Modifique o programa de contagem de cédulas para que ele trabalhe também com notas de R$ 100,00.

9. Modifique novamente o programa para aceitar valores decimais, ou seja, contar moedas de 0,01; 0,02; 0,05; 0,10 e 0,50.

10. O que acontece se digitarmos 0,001 no programa anterior? Caso ele não funcione, altere-o de forma a corrigir o problema.

11. Escreva um programa que leia um número e verifique se é ou não um número primo. Para fazer essa verificação, calcule o resto da divisão do número por 2 e depois por todos os números impares até o número lido. Se o resto de uma dessas divisões for igual a zero, o número não é primo. Observe que 0 e 1 não são primos e que 2 é o único número primo que é par.

12. Modifique o programa anterior de forma a ler um número n. Imprima os n primeiros números primos.

13. Escreva um programa que calcule a raiz quadrada de um número. Utilize o método de Newton para obter um resultado aproximado. Sendo n o número a obter a raiz quadrada, considere a base b=2. Calcule p usando a fórmula p=(b+(n/b))/2. Agora, calcule o quadrado de p. A cada passo, faça b=p e recalcular p usando a fórmula apresentada. Pare quando a diferença absoluta entre n e o quadrado de p for menor que 0,0001.